

Design and Analysis of Algorithms (CS2009)

Date: Nov,5th 2024

Course Instructor(s)

Dr.Muhammad Atif Tahir, Dr.Nasiruddin, Dr.Kamran,
Dr. Fahad Sherwani, Dr. Farrukh Salim, Ms. Ansum
Hamid,Mr.Syed Faisal Ali,Mr.Sandesh,Mr.Minhal
Raza ,Mr. Abu Zohran

Sessional-II Exam

Total Time (Hrs): 1.5

Total Marks: 17.5

Total Questions: 5

Roll No

Section

Student Signature

Do not write below this line

Attempt all the questions.

CLO #1: To apply acquired knowledge to solve computing problems complexities and proofs.

Q1 [3 Points]: Given the text $T = \text{"abcbcababcbcabcb"}$ and the pattern $P = \text{"abcbcab"}$, use the **Knuth-Morris-Pratt (KMP) algorithm modified version given below** to determine all occurrences of P in T . Show the working of KMP including the complete failure (prefix) table and perform a step-by-step dry run of the matching process. Document each pointer movement and skip, particularly when using the failure table after mismatches

Algorithm KMPMatchAll(T, P)

```
 $F \leftarrow \text{failureFunction}(P)$  // Preprocess pattern  $P$  to create the failure function  $F$  (Refer to Appendix)
 $i \leftarrow 0$  // Index for text  $T$ 
 $j \leftarrow 0$  // Index for pattern  $P$ 
 $n \leftarrow \text{length}(T)$  // Length of the text  $T$ 
 $m \leftarrow \text{length}(P)$  // Length of the pattern  $P$ 
 $\text{matches} \leftarrow []$  // List to store starting indices of matches
while  $i < n$ 
    if  $T[i] = P[j]$  then
        if  $j = m - 1$  then
             $\text{matches.append}(i - j)$  // Store the starting index of the match
             $j \leftarrow F[j]$  // Update  $j$  using the failure function for next potential match
        else
             $i \leftarrow i + 1$  // Move to the next character in the text
             $j \leftarrow j + 1$  // Move to the next character in the pattern
    else
        if  $j > 0$  then
             $j \leftarrow F[j - 1]$  // Use the failure function to find the next position in the pattern
        else
             $i \leftarrow i + 1$  // Move to the next character in the text
return matches // Return the list of starting indices of all matches
```

National University of Computer and Emerging Sciences

Karachi Campus

```
Algorithm failureFunction(P) // for Question-1
    F[0] = 0
    i = 1
    j = 0
    while i < m
        if P[i] = P[j]
            {we have matched j + 1 chars}
            F[i] = j + 1
            i = i + 1
            j = j + 1
        else if j > 0 then
            {use failure function to shift P}
            j = F[j - 1]
        else
            F[i] = 0 { no match }
            i = i + 1
```

CLO #4 : To construct and analyze real world problems solutions using different algorithms design techniques i.e. Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms.

Q2 [3 Points]: Design an algorithm that calculates the n-th Fibonacci number in linear time using dynamic programming. Clearly describe the main steps of Dynamic Programming for the above mentioned of nth Fibonacci series. You may design the algorithm using the main principles of Dynamic Programming including (i) the structure of an optimal solution (ii) Principle of Optimality (iii) Bottom Up Computation (iv) (Optional) Construction of any optimal solution. Remember, The Fibonacci sequence is the series of numbers where each number is the sum of the two preceding numbers. For example, 0, 1, 1, 2, 3, 5, 8 and so on

CLO #3 : To evaluate generic algorithmic solutions such as sorting, searching and graphs applied to real-world problems

Q3 [3 Points]: Create a list of tuples or pairs, where each tuple contains a player ID and their corresponding score. For example, player_scores = [(ID₁, score₁), (ID₂, score₂), ..., (ID_n, score_n)]. This is a multiplayer online game, where each player is assigned a unique score based on their performance. The game developers need to display the top 100 players' scores quickly and in linear time. The scores are represented as 32-bit integers ranging from 0 to 1,000,000. Design an algorithm (pseudocode) to sort and display the top 100 scores from a list of player scores. You are also required to design algorithm that use memory as efficiently as possible and consider implementing an additional step to keep track of the player IDs associated with each score. Explain your approach in pseudocode and the potential benefits of your modifications.

CLO #4 : To construct and analyze real world problems solutions using different algorithms design techniques i.e. Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms.

Q4 [3.5 Points]: Imagine you're packing for a trip and need to fit everything you want to bring into a suitcase. There's only so much room in your suitcase, so you can't pack items that would collectively exceed the available space. Airlines have weight restrictions on luggage, so the combined weight of your suitcase and its contents must not go over a specific limit.

With these limitations, you'll need to prioritize. Some items, like a laptop or passport, are essential and they must be part of the suitcase while others are non-essential items, like multiple pairs of shoes,

National University of Computer and Emerging Sciences

Karachi Campus

extra clothes, Camera, Shoes or Book, might be nice to have but less important. After fitting Laptop and Passport, you can only fit 8 kg of items in suitcase. The goal is to find non-essential items that provide the most "value" or importance to you for your trip, without exceeding your remaining luggage's weight limits of 8 kg per suitcase. For the above problem provide a solution based on dynamic programming bottom up approach to find out which pairs of non-essential items you are selected for packing the suitcase that gives you most importance value.

Item (i)	Weight (i)	Importance (i)
Book	1	1
Shoes	4	5
Camera	3	4
Clothes	5	7
Toilette's	2	3
Laptop	6	9
Passport	7	11

CLO #1 : To apply acquired knowledge to solve computing problems complexities and proofs.

Q5 [5 Points]: You are an engineer tasked with optimizing network coverage across several cell towers in a large region. The towers are represented by their coordinates on a 2D grid as follows:

1. Tower P: (7, 1)
2. Tower Q: (1, 7)
3. Tower R: (3, 4)
4. Tower S: (4, 3)
5. Tower T: (2, 5)
6. Tower U: (5, 2)

Complete the following

- (a) [1 Point] Sort all towers co-ordinates according to x-axis using Counting Sort? Show all steps clearly [1 Point]
- (b) [0.5 Points] What is the complexity using Computing Sort for the above mentioned scenario: $O(6)$, $O(13)$ or $O(12)$
- (c) [0.5 Points] Split the points with line L so that half the points are on each side. Consider the vertical line passing through the middle point (4,3). Plot all the points
- (d) [1 Point] Use City Block Distance and Brute Force Approach to compute distance of all points on the left hand side of the Line L and right hand side of the Line L.
Hint: Assume (6,7) and (13,2) are two points, city block distance is calculated as $|6-13| + |7-2| = 12$
- (e) [0.25 Point] What is the minimum distance d_l on left hand side and d_r on right hand side? Calculate the minimum of $d = \min(d_l, d_r)$
- (f) [0.5 Point] Find points close (closer than d) to the line passing through the middle point
- (g) [0.25 Point] Sort points obtained in (f) using y-coordinate. No need to apply any algorithm. Just manually sort these
- (h) [0.5 Point] Find pair of points with difference between y coordinates is smaller than d
- (i) [0.25 Point] Use Brute Force to compute distances b/w all points and assign minimum as d'
- (j) [0.25 Point] Which one is smaller d or d' and this is your final shorted pair using divide and conquer